

Roman to integer Conversion

Aim

To define a package named `number`, create a `Roman` class inside that package, implement the `romanToInteger()` method, and import/access that method from another class.

Input: "LVIII" **Output:** 58 **Explanation:** L = 50, V = 5, III = 3, so 50 + 5 + 3 = 58

System Requirements

Hardware

Standard PC / Laptop
Intel Core i3 / i5 processor or equivalent
Minimum 4 GB RAM
Minimum 500 MB free storage

Software

Operating System : Windows / Linux / macOS
Java Development Kit : JDK 17 or JDK 21
Text Editor or IDE : Eclipse, IntelliJ IDEA, NetBeans, or VS Code
Command Prompt / Terminal

Source Code

```
package number;

import java.util.Scanner;

import static number.Roman.romanToInteger;

public class Roman {

    // Convert the Roman number into an integer.
    public static int romanToInteger(String romanNumber) {
        if (romanNumber == null || romanNumber.isEmpty()) {
```

```

        throw new IllegalArgumentException("Roman number
cannot be empty");
    }

    int result = 0;
    for (int index = 0; index < romanNumber.length();
index++) {
        int currentValue =
valueOf(romanNumber.charAt(index));
        // Subtract when a smaller value comes before a
bigger value.
        if (index + 1 < romanNumber.length()
            && currentValue <
valueOf(romanNumber.charAt(index + 1))) {
            result -= currentValue;
        } else {
            result += currentValue;
        }
    }
    return result;
}

private static int valueOf(char romanNumber) {
    switch (romanNumber) {
        case 'I':
            return 1;
        case 'V':
            return 5;
        case 'X':
            return 10;
        case 'L':
            return 50;
        case 'C':
            return 100;
        case 'D':
            return 500;
    }
}

```

```

        case 'M':
            return 1000;
        default:
            throw new IllegalArgumentException("Invalid
Roman number: " + romanNumber);
    }
}

class RomanToInteger {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Get the Roman number from the user.
        System.out.print("Enter a Roman number: ");
        String romanNumber = scanner.nextLine();

        // Call the method from the Roman class.
        int integerValue = romanToInteger(romanNumber);
        System.out.println("Integer value: " + integerValue);

        scanner.close();
    }
}

```

A **package** is a folder that groups related Java classes together. It prevents name clashes and organizes code.

Defining a package	->	package number;	(first line of the file)
Creating a package	->	folder named 'number'	holding the .java files
Accessing a package	->	import number.Roman;	(bring the class in)
Static import	->	import static number.Roman.romanToInteger;	(bring one static METHOD in, so it can be called directly without writing Roman.)

Here the package is `number`, the class inside it is `Roman`, and the method `romanToInteger()` is made `public static` so another class can use it.

Exception used — `IllegalArgumentException`

`IllegalArgumentException` is an **unchecked (runtime) exception** thrown when a method receives an argument that is invalid for its logic. It extends `RuntimeException`, so the compiler does **not** force you to catch it.

In this program it is thrown in two places:

Where	Condition	Message
<code>romanToInteger()</code>	input is null or empty	Roman number cannot be empty
<code>valueOf()</code>	character is not I, V, X, L, C, D, M	Invalid Roman number: <char>

Note: `main()` does **not** catch this exception. If the user enters an invalid symbol like "ABC", the program will crash with a stack trace. To handle it gracefully, wrap the call in a try-catch (`IllegalArgumentException e`) block.

2. Step-wise Execution

1. File begins with "package number;" – the class belongs to package 'number'.
2. The static import makes `romanToInteger()` usable by its short name.
3. Program starts in `RomanToInteger.main()`.
4. Scanner object is created to read input.
5. User is prompted: "Enter a Roman number: " and types e.g. LVIII.
6. `romanToInteger("LVIII")` is called.
7. Inside `romanToInteger()`:
 - a. IF input is null or empty -> throw `IllegalArgumentException`.
 - b. result is set to 0.
 - c. A for loop runs through every character:

```
index 0 -> 'L' = 50
           next char 'V' = 5 ; 50 is NOT less than 5
           -> ADD      result = 0 + 50 = 50
index 1 -> 'V' = 5
           next char 'I' = 1 ; 5 is NOT less than 1
           -> ADD      result = 50 + 5 = 55
index 2 -> 'I' = 1
           next char 'I' = 1 ; 1 is NOT less than 1
           -> ADD      result = 55 + 1 = 56
index 3 -> 'I' = 1
           next char 'I' = 1 ; 1 is NOT less than 1
           -> ADD      result = 56 + 1 = 57
index 4 -> 'I' = 1
           no next char
           -> ADD      result = 57 + 1 = 58
```
 - d. result = 58 is returned.
8. "Integer value: 58" is printed.

9. Scanner is closed.

SUBTRACTION RULE EXAMPLE – input "IX":

```
index 0 -> 'I' = 1
           next char 'X' = 10 ; 1 IS less than 10
           -> SUBTRACT result = 0 - 1 = -1
index 1 -> 'X' = 10
           no next char
           -> ADD      result = -1 + 10 = 9
Answer = 9
```

3. Algorithm

START

PART A – valueOf(char)

1. SWITCH on the character:

'I' -> RETURN 1

'V' -> RETURN 5

'X' -> RETURN 10

'L' -> RETURN 50

'C' -> RETURN 100

'D' -> RETURN 500

'M' -> RETURN 1000

DEFAULT -> THROW IllegalArgumentException("Invalid Roman number")

PART B – romanToInteger(String)

2. IF input is null OR empty:

THROW IllegalArgumentException("Roman number cannot be empty")

3. SET result = 0

4. FOR index = 0 TO length - 1:

a. currentValue = valueOf(character at index)

b. IF index + 1 < length AND

currentValue < valueOf(character at index + 1):

result = result - currentValue // subtraction rule

ELSE:

result = result + currentValue // addition rule

5. RETURN result

PART C – main()

6. Create Scanner object

7. Prompt and read Roman number from user

8. Call romanToInteger(romanNumber)

9. Print "Integer value: " + returned value

10. Close Scanner

STOP

4. UML Class Diagram

- Both classes live inside the **package number** (shown by the enclosing package box).
- `Roman` exposes one **public static** method, `romanToInteger()`; `valueOf()` is **private** (–) and used only internally — this is **encapsulation**.
- `RomanToInteger` **uses** `Roman` through a static import (a *dependency*, not inheritance).
- Both methods can **throw** `IllegalArgumentException`, which extends `RuntimeException`.

Compilation and Execution

Folder structure:

```
project/  
├── number/  
│   └── Roman.java
```

Compile:

```
javac number/Roman.java
```

Run:

```
java number.RomanToInteger
```

Sample Output

Enter a Roman number: LVIII

Integer value: 58

Enter a Roman number: IX

Integer value: 9

Enter a Roman number: MCMXCIV

Integer value: 1994

Output:

```
PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'number.RomanToInteger'  
Enter a Roman number: XLIXIV  
Integer value: 954  
PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'number.RomanToInteger'  
Enter a Roman number: LMXADIVX  
Integer value: 1439  
PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'number.RomanToInteger'  
Enter a Roman number: XLDCMLIIIVIX  
Integer value: 904  
PS C:\Users\Admin\OneDrive\Documents\basicjava>
```

Result

The Roman-to-Integer conversion program was successfully implemented in Java using the concept of packages. A user-defined package `number` was created, the `Roman` class was defined inside it with the `romanToInteger()` method, and the method was successfully imported and accessed from another class using a static import. The program correctly applies the Roman numeral addition and subtraction rules, converting the input "LVIII" to the output 58, and validates invalid input using `IllegalArgumentException`.